COMP517 – Data Analysis

Lab 2 –Introduction to NumPy and pandas

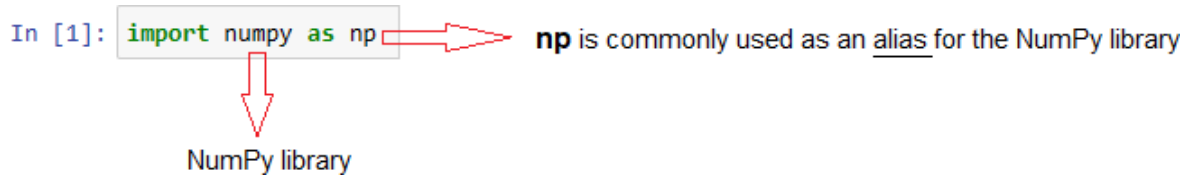
In this lab, you will be introduced to the basics of NumPy arrays and pandas DataFrames. By the conclusion of this lab, you will have a foundational understanding of NumPy arrays and pandas DataFrames, along with their associated functions.

Tips & Advice For tips and advice see Appendix A provided at the end of this document.

NumPy Library

NumPy is an open-source library and an essential component for numerical computing in Python. It stands for "Numerical Python" and provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays efficiently.

NumPy Installation and Import NumPy can be installed on a PC using `pip` or `brew`. After the installation is successful, you must import NumPy. For users using Jupyter Notebook with an Anaconda distribution or any other Python distribution that includes NumPy by default, installing NumPy separately is unnecessary. All you need to do is directly import it.



In Python, you can save typing and make the code more concise and readable by using **np** as the alias. Using np as the alias for NumPy is not mandatory, and you can choose any other alias you prefer. However, np has become a widely adopted and recognized convention in the Python data science community, making it easier for others to understand your code.

NumPy Arrays

In lab 01, you were introduced to Python lists, a built-in data structure used to store a collection of elements. NumPy arrays are also used to store collections of elements, but they have major differences with Python lists in terms of functionality, performance, and use cases. In this lab, we explore different syntaxes to create NumPy arrays with homogeneous data types. NumPy arrays are homogeneous, meaning all elements must have the same data type. This allows for efficient memory storage and optimized mathematical operations. In contrast, Python lists can contain elements of different data types, making them less memory-efficient and slower for certain operations. Possible data types are bool, int, float, long, double, and long double. Some more differences are provided in Appendix B: Part A/Table 1.

There are different ways to create NumPy arrays, such as one-dimensional arrays, two-dimensional arrays (matrices), arrays filled with zeros or ones, identity matrices, random arrays, and range arrays.

1. Create NumPy arrays: One-dimensional and multi-dimensional

- Create a One-dimensional **array** (vector)

```
array_1d = np.array([10, 20, 30, 40, 50])
print("One-dimensional array:")
print(array_1d)
# Output: [10 20 30 40 50]
```

This is a one-dimensional array with 5 elements.

- Create a Two-dimensional **array** (matrix)

```
array_2d = np.array([[10, 20, 30], [40, 50, 60]])
# notice the outer square brackets
print("Two-dimensional array:")
print(array_2d)
# Output:
# [[10 20 30]
#  [40 50 60]]
```

This is a two-dimensional array with 2 rows and 3 columns.

- Create an array of **ones**: Run the below code to create a 2x3 array filled with ones.

```
ones_array = np.ones((2, 3))
print("Array of ones:")
print(ones_array)
```

- Create a Two-dimensional float array (matrix): Run the below code to create a two-dimensional float array with 2 rows and 3 columns.

```
# Two-dimensional float array (matrix)

matrix_float = np.array([[100, 2.2, 3.3], [4.4, 5.5, 6.6]])

print("Two-dimensional float array (matrix):")

print(matrix_float)

# Output:

#Two-dimensional float array (matrix)

#[[100.    2.2   3.3]
 #[  4.4   5.5   6.6]]
```

If you want to show the data type of the elements inside the matrix_float array (rather than the data type of the whole array), you can use the **dtype** attribute of the NumPy array:

```
print(matrix_float.dtype)
```

- Create Identity matrix: Run the below code to create a 3x3 identity matrix with ones on the main diagonal and zeros elsewhere.

```
identity_matrix = np.eye(3)

print("Identity matrix:")

print(identity_matrix)
```

- **Create a Random array:** The **np.random.rand()** function is a part of the NumPy library, which is used for generating random numbers with a uniform distribution between 0 and 1. The syntax of this function is presented as:

```
np.random.rand(d0, d1, ..., dn)
```

where **d0, d1, ..., dn** are integers representing the dimensions of the output array.

- Complete the below code to create a Three-dimensional array (2x3x4) filled with random values between 0 and 1. Provide and study the output.

```
# Three-dimensional array (2x3x4)

array_3d = np.random.rand(,,) #To be completed

print("Three-dimensional array")

print() #To be completed
```

❖ The **np.arange()** function in NumPy is used to create an array with regularly spaced values within a specified range. The syntax of **np.arange()** is as follows:

```
np.arange(start, stop, step)
```

where

- **start**: The start of the range (inclusive). The array will start with this value.
- **stop**: The end of the range (exclusive). The array will stop just before reaching this value.
- **step**: The spacing between values in the array. It represents the difference between consecutive values.

Example: The **np.arange(0, 10, 2)** call creates a NumPy array with values from 0 to 10 (exclusive) with a step of 2.

- The array starts at **0** (inclusive) and ends just before **10** (exclusive).
- The values in the array are generated with a step of **2**, meaning there is a difference of **2** between consecutive elements.
- The resulting array contains the values **[0, 2, 4, 6, 8]**.

➢ Complete the below code to generate an array of floating-point numbers from 0.5 to 5.5 (exclusive) with a step of 0.5 between consecutive elements.

```
#Using np.arange() with floating-point numbers
array_frange = np.arange(, ,) #To be completed
print()#To be completed
```

2. Mathematical Operation on NumPy Arrays

NumPy arrays allow you to perform various mathematical operations and work with multidimensional data structures efficiently.

2.1. Mathematical Operation

NumPy arrays allow you to perform element-wise operations easily. For example, you can add, subtract, multiply, or divide arrays element by element without the need for explicit loops.

In the below examples, we create two one-dimensional arrays (array1 and array2) to perform Additive, Multiplication, and Exponentiation operations on them:

```
# Create two NumPy arrays
array1 = np.array([1, 2, 3, 4, 5])
array2 = np.array([10, 20, 30, 40, 50])

# Addition
result_add = array1 + array2
print(result_add)
# Output: [11 22 33 44 55]

# Multiplication
result_multiply = array1 * array2
print(result_multiply)
# Output: [ 10  40  90 160 250]

# Exponentiation on array1
result_power = array1 ** 2
print(result_power)
# Output: [ 1  4  9 16 25]
```

- Complete the below syntaxes to perform Subtraction, Division, and Element-wise square root operations on array1 and array2. Provide the output.

```
# Subtraction
result_sub = #To be completed
print()#To be completed
# Division
result_div = #To be completed
print()#To be completed


''' Element-wise square root on the array of your choice. Hint: use NumPy's
sqrt() function '''
result_sqrt = #To be completed
print()#To be completed
```

Note: Using ''' ''' (triple quotes) for multi-line comments can be helpful when you need to add more extensive explanations or context to your code without cluttering it with multiple single-line comments.

2.2. Statistical Operations

The NumPy library provides many functions for various statistical computations, making it a powerful tool for data analysis and manipulation in Python. In the below examples, we create one-dimensional arrays (numbers) to perform a few statistical operations on them. You will learn more about statistical computations and their interpretation in data analysis in future lectures and labs.

```
# First Create a NumPy array
numbers = np.array([12, 7, 9, 15, 8, 13])

# Maximum and Minimum: Find the maximum and minimum numbers in the data
max_value = np.max(numbers)
min_value = np.min(numbers)
print("Maximum:", max_value)
print("Minimum:", min_value)
# Output: Maximum: 15
#         Minimum: 7

# Mean: Calculate the average of the data and round it to 2 decimal places
mean_value = np.mean(numbers)
rounded_mean = round(mean_value, 2)
print("Mean:", rounded_mean)
# Output: Mean: 10.67

# Sum: Calculate the sum of all elements in the numbers
sum_value = np.sum(numbers)
print(sum_value)
# Output: 64
```

2.3. Multidimensional Data Structures:

Multidimensional data structures in NumPy are represented by arrays with multiple dimensions, commonly known as ndarrays. These arrays can have two or more dimensions, allowing you to work with data in a matrix-like format and handle more complex data structures efficiently. This versatility makes NumPy an essential tool for scientific computing and data analysis in Python.

In the below examples, we first create a 2D NumPy array from a Python list of lists! and name it matrix.

```
# Create a 2D NumPy array
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
In [72]:
''' The matrix will look like below when you call it:
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]]) '''
matrix
```

```
Out[72]: array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

```
# Accessing elements in the 2D array
print(matrix[0, 1])
# Output: 2 (element at first row (index 0), and second column (index 1))
print(matrix[2, 0])  # Output: 7 (element at row 2, column 0)
```

- Complete the below syntaxes to access the elements in the second row and third column. Provide the output.

```
element_1_2 = matrix[,]  #To be completed
print("Element at (1, 2):",) #To be completed
```

- You can perform slicing on the ndarray. In the below code, we attempt to access the second column of the matrix:

```
second_column = matrix[:, 1]  # Accessing the second column
print("Second Column:", second_column)
# Output: Second Column: [2 5 8]
```

- Complete the below syntaxes to access the second row. Provide the output.

```
second_row= matrix[,]  #To be completed
print("second Row: ",) #To be completed
```

We can perform mathematical operations on a nd NumPy array. Let's print the original matrix first:

```
In [77]:
1 # Printing the original matrix
2 print("Original Matrix:")
3 print(matrix)

Original Matrix:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

We can add/subtract or perform element-wise division/ multiplication/square root etc:

```
# Adding a scalar value to the matrix
matrix_add_scalar = matrix + 10
print("Matrix + 10:")
print(matrix_add_scalar)


# Element-wise division by a scalar
matrix_divide_scalar = matrix / 2
print("Matrix / 2:")
print(matrix_divide_scalar)
```

In the below example, a second 2D NumPy array (matrix2) is created. Then, the + operator is used to perform element-wise addition of the two matrices, resulting in the `matrix_sum`. Each element in `matrix_sum` is obtained by adding the corresponding elements from matrix and matrix2.

```
# Element-wise addition of two matrices
matrix2 = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
matrix_sum = matrix + matrix2
print("Matrix Sum:")
print(matrix_sum)
```

Matrix multiplication is a crucial operation in many fields, including linear algebra, computer graphics, data science, and machine learning. NumPy's efficient implementation allows for fast computation of matrix products, making it a powerful tool for working with multidimensional data in Python. For example, the dot product is utilized in data sciences and machine learning such as linear regression, Neural Networks (NN), image processing, Singular Value Decomposition (SVD), etc. See Appendix B Part B for the visual representation of matrix multiplication.

```
# Matrix multiplication (dot product)
matrix3 = np.array([[2], [3], [4]])
matrix_dot_product = np.dot(matrix, matrix3)
# Alternatively, you can use the @ operator for matrix multiplication
# matrix_dot_product = matrix @ matrix3
print("Matrix Dot Product:")
print(matrix_dot_product)
```

Pandas library

pandas, a widely-used open-source Python library, is named after "panel data," an econometric term, and "Python data analysis." It is worth noting that the official pandas documentation advocates using lowercase letters when referring to the project's name.

pandas Installation and Import

Jupyter Notebook relies on the Python kernel associated with your notebook, which should include all the installed packages accessible in your Python installation. If you have pandas installed in the Python environment that your Jupyter Notebook is using, you can directly import and use it in your notebook without any additional installation steps. Otherwise, use pip to install it before import:

```
1 !pip install pandas
```

To import pandas run the below import statement in a Jupyter Notebook cell:

```
1 import pandas as pd
```

If there are no error messages displayed after running the code, it means pandas is successfully imported with the alias **pd**. The alias **pd** is a common convention used when importing the pandas library. The choice of using **pd** as the alias is optional, but it has become a standard in the pandas community, and is often used in pandas-related documentation, tutorials, and examples.

DataFrame

'DataFrame' is one of the core data structures in pandas making it a fundamental tool for data analysis and manipulation in Python. It represents tabular data in a 2-dimensional tabular format, similar to a spreadsheet or SQL table. A DataFrame can hold data of different types (e.g., integers, strings, floating-point numbers) and allows you to perform various data operations efficiently.

Create a DataFrame You can create a DataFrame from various data sources such as dictionaries, lists, NumPy arrays, CSV files etc.,

Let's create a DataFrame (named **df)** from a dictionary (named **people)**, where each key-value pair represents a column.

To start with, create a dictionary. The dictionary **people** contains three keys: **'Name'**, **'Age'**, and **'City'**.

```
people = {
    'Name': ['John', 'Alice', 'Bob'],
    'Age': [25, 30, 22],
    'City': ['New York', 'San Francisco', 'Los Angeles']
}
```

The below code creates a DataFrame(df) from people.

```
df = pd.DataFrame(people)
```

- **pd** is the alias used to refer to the pandas library.
- **pd.DataFrame()** is a constructor provided by pandas to create a DataFrame from various data sources(here a dictionary)
- Inside the **pd.DataFrame()** constructor, we pass the **people** dictionary as an argument. Each key maps to a list of values that represent the data for each column in the DataFrame.
- Each key in the dictionary becomes a column label, and the corresponding list of values becomes the data in that column.

By running the below code, the DataFrame is displayed as a tabular structure, with rows and columns representing the data points and attributes, respectively.

```
# Displaying the DataFrame
print(df)
```

```
    Name  Age           City
0   John   25       New York
1  Alice   30  San Francisco
2    Bob   22    Los Angeles
```

The **Name**, **Age**, and **City** are the column labels, and each row represents a data record with the corresponding values for each column. As you can see, the DataFrame has three rows. The numbers 0, 1, and 2 displayed on the leftmost column of the output are the index labels assigned to each row.

Note: The default integer-based index makes it easier to access individual rows or perform operations based on their positions in the DataFrame. You can use **.iloc** to access rows by their integer-based index:

```
# Accessing the first row (index 0)
row_0 = df.iloc[0]
print(row_0)
```

```
Name         John
Age            25
City     New York
Name: 0, dtype: object
```

DataFrame contains columns with mixed data types (string and integer). Here, pandas used the **object** data type to accommodate the heterogeneity. Use the **.dtypes** attribute of the DataFrame to check the data types of each column.

```
print(df.dtypes)
```

- Provide and study the output of the above code.

Now that you have a DataFrame, you can perform various data operations, such as selecting specific rows or columns, filtering data based on conditions, performing arithmetic operations, and applying functions to the data.

You can access individual columns using square brackets and the column name as a key.

Complete the below code to print people's names stored in the DataFrame df.

```
# Access the 'Name' column
name_column = df[] #To be completed
print() #To be completed
```

Individual rows can be accessed using **.iloc[]** by providing the row index. Complete the below code to print the last row of the DataFrame df:

```
# Access the last row using the last row (index -1)
last_row = df.iloc[] #To be completed
print() #To be completed
```

A DataFrame can be filtered based on certain conditions. Let's filter rows where the age is greater than 25:

```
filtered_data = df[df['Age'] > 25]
```

1. **df['Age'] > 25**: Creates a boolean Series that represents the comparison result of each value in the **'Age'** column with the value 25. For each element in the **'Age'** column, it will return **True** if the age is greater than 25, and **False** otherwise.
2. **df[df['Age'] > 25]**: This part uses boolean indexing to filter the rows in the DataFrame **df**. It keeps only the rows where the corresponding value in the **'Age'** column is greater than 25. In other words where the boolean Series is **True**.
3. Finally, a new DataFrame named **filtered_data** stores the filtered rows where the age is greater than 25.

You can perform aggregation operations on the DataFrame, like calculating the mean, sum, etc. Aggregation operations in pandas allow you to perform calculations that summarize or reduce data in a DataFrame. They are useful for obtaining insights and statistics from your data.

Let's calculate the sum of ages using the sum() function:

```
# Calculate the sum of ages
total_age = df['Age'].sum()
print("Total Age:", total_age)
```

- Now you complete the below code to get and print the minimum and maximum age using the min() and max() functions respectively:

```
# Calculate the minimum and maximum age
min_age = df['Age'].  #To be completed
max_age = df['Age']. #To be completed

print("Minimum Age:",) #To be completed
print("Maximum Age:",) #To be completed
```

- Run the below code. Explain the code and the output:

```
city_mean_age = df.groupby('City')['Age'].mean()
print(city_mean_age)
```

To add a new row to a DataFrame in pandas, you can use the **append()** method as below. Note that we can only append a dictionary if ignore_index=True.

```
# Create a dictionary with the data for the new row
new_row = {
    'Name': 'Eve',
    'Age': 28,
    'City': 'Chicago'
}
# Convert the list of dictionaries to a DataFrame

new_row_df=pd.DataFrame([new_row])

# Use concat to add the new rows to the DataFrame

df = pd.concat([df,new_row_df],ignore_index=True)

print(df)
```

Q: What if we don't know the Age or City of a person?

A: By using **np.nan**, you can explicitly insert null values in a DataFrame when dealing with missing or unknown data. There are alternative solutions as well.

```
# List of dictionaries representing the data for multiple new rows
new_rows = [
    {'Name': 'Carlos', 'Age': None, 'City': 'Chicago'},
    {'Name': 'Michael', 'Age': 33, 'City': np.nan},
    {'Name': 'Sophia', 'Age': np.nan, 'City': np.nan}
]
# Convert the list of dictionaries to a DataFrame

new_rows_df = pd.DataFrame(new_rows)

# Use concat to add the new rows to the DataFrame

df = pd.concat([df, new_rows_df],
ignore_index=True)

print(df)
```

The output will look like below showing the missing value in your DataFrame.

```
      Name   Age           City
0     John  25.0       New York
1    Alice  30.0  San Francisco
2      Bob  22.0    Los Angeles
3   Carlos   NaN        Chicago
4  Michael  33.0            NaN
5   Sophia   NaN            NaN
```

Note: Using **np.nan** / **None** to represent missing values in a pandas DataFrame changed the data type of the column containing missing values to float, even if the column originally had integer values. This is because **NaN** is a special floating-point value that represents missing data, and pandas automatically converts integer values to floating-point when **NaN** is introduced.

To get the shape of a pandas DataFrame **df**, you can use the **.shape** attribute. The **.shape** attribute returns a tuple representing the dimensions of the DataFrame, where the first element is the number of rows (axis 0) and the second element is the number of columns (axis 1).

```
# Get the shape of the DataFrame

shape = df.shape

print("Shape of the DataFrame:", shape)
```

DataFrame **df** has 6 rows and 3 columns. The shape of the DataFrame is returned as a tuple **(6, 3)**, indicating that the DataFrame has 6 rows (axis 0) and 3 columns (axis 1).

In the real world, the DataFrame might hold hundreds of rows, and printing the DataFrame for visually inspecting the missing value is not ideal! Instead, you can check for missing values using the **isnull()** function and then count the number of missing values in each column.

```
# Check for missing values using isnull()

missing_values = df.isnull()


# Count the number of missing values in each column

missing_counts = missing_values.sum()


print("Number of Missing Values in Each Column:")

print(missing_counts)
```

The `missing_values` hold the output of isnull(), where each element corresponds to whether the corresponding element in the original DataFrame is null (**True**) or not null (**False**). You can inspect the `missing_values` DataFrame using print function (`print(missing_values)`)

Using **isnull()** function in combination with other pandas functions like **sum()**, **count()**, **fillna()**, or **dropna()**, allows you to efficiently handle missing data, perform data imputation, or remove rows/columns with missing values based on your data analysis needs.

Complete the below code to fill in the missing values in the 'Age' column of the df DataFrame with the average age.

```
# Calculate the mean of the 'Age' column
average_age = df['Age'].       #to be completed


''' Use fillna() to replace missing values in the 'Age' column with the
average age '''
df['Age'] = df['Age'].fillna()     #to be completed
print(df)
```

```
      Name   Age           City
0     John  25.0       New York
1    Alice  30.0  San Francisco
2      Bob  22.0    Los Angeles
3   Carlos  27.5        Chicago
4  Michael  33.0            NaN
5   Sophia  27.5            NaN
```

Using **dropna()** allows you to clean the DataFrame by removing rows with missing values, which is often necessary before performing certain data analyses or when missing values cannot be imputed effectively.

```
# Drop rows with missing values in the 'City' column
df_dropped_city = df.dropna(subset=['City'])


print(df_dropped_city)
```

```
     Name   Age           City
0    John  25.0       New York
1   Alice  30.0  San Francisco
2     Bob  22.0    Los Angeles
3  Carlos  27.5        Chicago
```

Use the **.shape** attribute to get the new dimension of your DataFrame df.

- ✓ You will learn more about missing value imputation methods in future lectures and labs.

Read from/Save to CSV files:

The input/output (I/O) section of the pandas library enables users to read data from various sources and write data back to different formats. Pandas provides a wide range of functions and methods to handle data in different file formats such as Excel, SQL databases, JSON, etc more efficiently. To explore the

full documentation and examples related to the input/output capabilities of pandas, you can visit the official Pandas website.

To read data from a **CSV** (Comma-Separated Values) file and create a pandas DataFrame, you can use the **pd.read_csv()** function from pandas. Download the 'Lab02Data.csv' file and save it under the same directory as your Python file.

```
# Read data from the CSV file and create a DataFrame

mydf = pd.read_csv('Lab02Data.csv')

# Display the DataFrame

print(mydf)

#Number of rows and columns

mydf.shape
```

Let's drop two rows:

	Age	Name	City
0	30	Luke Skywalker	Gondor
1	20	Obi-Wan Kenobi	Rivendell
2	27	Leia Organa	Rohan
3	38	Chewbacca	Hobbiton
4	38	Darth Vader	Mirkwood
5	29	Rey	Mirkwood
6	58	Chewbacca	Lothlórien
7	53	R2-D2	Gondor
8	46	Chewbacca	Minas Tirith
9	56	Kylo Ren	Edoras
10	53	Kylo Ren	Rivendell
11	28	Obi-Wan Kenobi	Rivendell
12	52	R2-D2	Bree
13	42	Kylo Ren	Isengard
14	59	Rey	Rohan
15	24	R2-D2	Minas Tirith
16	21	Yoda	Hobbiton
17	20	Chewbacca	Bree
18	32	Luke Skywalker	Edoras
19	37	Rey	Hobbiton

```
# Drop rows with index labels 10 and 15

mydf.drop(index=[10, 15], inplace=True)

print(mydf)

# What does the inplace mean here?
```

	Age	Name	City
0	30	Luke Skywalker	Gondor
1	20	Obi-Wan Kenobi	Rivendell
2	27	Leia Organa	Rohan
3	38	Chewbacca	Hobbiton
4	38	Darth Vader	Mirkwood
5	29	Rey	Mirkwood
6	58	Chewbacca	Lothlórien
7	53	R2-D2	Gondor
8	46	Chewbacca	Minas Tirith
9	56	Kylo Ren	Edoras
11	28	Obi-Wan Kenobi	Rivendell
12	52	R2-D2	Bree
13	42	Kylo Ren	Isengard
14	59	Rey	Rohan
16	21	Yoda	Hobbiton
17	20	Chewbacca	Bree
18	32	Luke Skywalker	Edoras
19	37	Rey	Hobbiton

To save the DataFrame with dropped columns or rows as a new CSV file, you can use the **to_csv()** method. This method allows you to save the DataFrame to a CSV file with the specified filename.

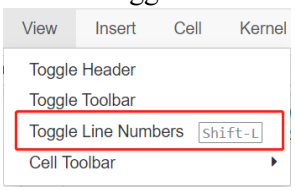
The **`index=False`** parameter ensures that the index column is not included in the saved CSV file. The 2Rowsdata_dropped.csv is now saved under the same directory.

```
# Save the modified DataFrame to a new CSV file
mydf.to_csv('2Rowsdata_dropped.csv', index=False)
print("Modified DataFrame saved to '2Rowsdata_dropped.csv' file.")
```

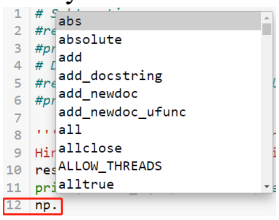
Appendix A

Tips and Advise

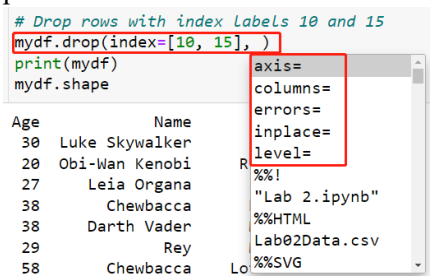
1. You can toggle line numbers from the View menu. It is useful for debugging.



2. Press the Tab button after typing 'np.' to see a list of available functions to use in the NumPy library.



3. Similarly, press the Tab button within the round bracket in a function to see a list of available parameters to enter.



4. Python (and almost any other programming language) is case-sensitive. **`True`** and **`False`** are not the same as `true` and `false`.

5. **Never** name a variable using a function name or a reserved keyword, such as print, and int.

Appendix B

Part A: Differences Between NumPy Arrays and Python Lists

Table 1: Differences Between NumPy Arrays and Python Lists.

	NumPy arrays	Python lists
Libraries and Environment	NumPy arrays are a part of the larger scientific computing environment in Python, including libraries like SciPy, pandas, and scikit-learn.	Python lists are more general-purpose and widely used in a variety of Python applications.
Performance	NumPy arrays are more efficient and faster for numerical computations due to their underlying C implementation and optimized algorithms.	Python lists are slower for numerical operations since they are interpreted and involve more overhead.
Functionality	NumPy arrays support a wide range of mathematical operations, broadcasting, and vectorized operations. They are suitable for scientific and numerical computations.	Python lists have limited mathematical functionality and require explicit loops for element-wise operations.
Memory Efficiency	NumPy arrays are more memory-efficient compared to Python lists, especially for large datasets, as they use contiguous memory blocks for homogeneous data types.	Python lists have additional memory overhead due to the flexible data type support and pointers for each element.
Syntax	NumPy arrays have a different syntax for creation and manipulation compared to Python lists. NumPy arrays can be created using *numpy.array()* or other functions like *numpy.zeros(), numpy.ones()*, etc.	Python lists are created using square brackets [].
Ease of Use	NumPy arrays are more suitable for complex mathematical operations and multidimensional data structures.	Python lists are generally easier to work with for general-purpose tasks and simple data structures.
Dimensionality	NumPy arrays can be multi-dimensional (e.g., 1D, 2D, 3D, etc.), making them useful for representing matrices and multi-dimensional data.	Python lists are one-dimensional and can be nested to represent multi-dimensional data, but accessing elements in nested lists can be less efficient.

Part B: Matrix Multiplication

```
A (m x n)   *   B (n x p)  =  C (m x p)

[ a11  a12  a13 ]      [ b11  b12 ]       [ c11  c12 ]
[ a21  a22  a23 ]   *  [ b21  b22 ]   =   [ c21  c22 ]
[ a31  a32  a33 ]      [ b31  b32 ] -->   [ c31  c32 ]


The elements c11, c12, c21, c22, c31, c32 in the resulting matrix C are obtained as follows:

c11 = a11 * b11 + a12 * b21 + a13 * b31
c12 = a11 * b12 + a12 * b22 + a13 * b32
c21 = a21 * b11 + a22 * b21 + a23 * b31
c22 = a21 * b12 + a22 * b22 + a23 * b32
c31 = a31 * b11 + a32 * b21 + a33 * b31
c32 = a31 * b12 + a32 * b22 + a33 * b32

Note that matrix multiplication is not commutative and the order of the matrices matters:

A * B ≠ B * A
```